

MongoDB - Node.js Integration Report

In this report, we cover the questions and topics discussed related to MongoDB, including using the MongoDB Native Driver and Mongoose for connecting to the database from a Node.js application.

Q1: What is MongoDB and how is it different from SQL databases?

MongoDB is a NoSQL database that stores data in JSON-like documents. Unlike SQL databases, MongoDB is schema-less, making it more flexible and scalable. While SQL databases like MySQL store data in tables with predefined schemas, MongoDB stores data in collections and allows for dynamic schemas.

Q2: How do we connect to MongoDB in a Node.js application?

To connect to MongoDB in a Node.js application, we use the MongoDB Native Driver or Mongoose.

Below are examples for both:

Native MongoDB Driver Connection Example:

```
const { MongoClient } = require('mongodb');

const url = 'mongodb://localhost:27017';

const dbName = 'myDatabase';

let db;

async function connectMongo() {

  const client = new MongoClient(url, { useUnifiedTopology: true });

  await client.connect();

  console.log("MongoDB connected!");

  db = client.db(dbName);

}

connectMongo();
```

Mongoose Connection Example:

```
const mongoose = require('mongoose');

const url = 'mongodb://localhost:27017/myDatabase';

mongoose.connect(url, { useNewUrlParser: true, useUnifiedTopology: true })

  .then(() => console.log('Mongoose connected!'))

  .catch((err) => console.log('Mongoose connection error: ', err));
```

Q3: How do we define a model and schema in Mongoose?

In Mongoose, schemas define the structure of the documents within a collection, and models allow us to interact with the database. Below is an example:

```
const mongoose = require('mongoose');

const Schema = mongoose.Schema;

// Define a schema

const userSchema = new Schema({

  name: { type: String, required: true },

  email: { type: String, required: true },

  age: { type: Number, required: true },

});

// Create a model

const User = mongoose.model('User', userSchema);

// Save a document using the model

const userData = new User({

  name: 'John Doe',
```

```
email: 'john@example.com',  
age: 28  
});  
  
userData.save()  
  .then((result) => console.log('User saved:', result))  
  .catch((err) => console.log('Error saving user:', err));
```

Q4: How to insert data into a collection using the native MongoDB driver?

In the native MongoDB driver, we can insert documents into a collection like this:

```
const { MongoClient } = require('mongodb');  
  
const url = 'mongodb://localhost:27017';  
  
const dbName = 'myDatabase';  
  
async function insertUser() {  
  const client = new MongoClient(url, { useUnifiedTopology: true });  
  await client.connect();  
  const db = client.db(dbName);  
  const result = await db.collection('users').insertOne({ name: 'Jane Doe', email:  
'jane@example.com' });  
  console.log('User inserted:', result);  
}  
  
insertUser();
```

Q5: How to insert data into a collection using Mongoose?

With Mongoose, the process is similar but we use a model:

```
const mongoose = require('mongoose');
```

```
const User = mongoose.model('User', userSchema);

const userData = new User({ name: 'Emily Doe', email: 'emily@example.com' });

userData.save()

  .then((result) => console.log('User saved:', result))

  .catch((err) => console.log('Error saving user:', err));
```

Conclusion:

This report covers the essential MongoDB connection methods with Node.js, using both the native MongoDB driver and Mongoose. We also discussed the process of connecting to MongoDB, defining models and schemas, and inserting documents into collections. These concepts are fundamental for building applications that leverage MongoDB as the data store.